Figure 21.4 The default filter ordering within a given stage, based on the scope of the filters. For the `*Executing` method, globally scoped filters run first, followed by controller-scoped, and finally action-scoped filters. For the `*Executed` method, the filters run in reverse order.

By default, filters execute from the broadest scope (global) to the narrowest (action) when running the `*Executing` method for each stage. The filters' `*Executed` methods run in reverse order, from the narrowest scope (action) to the broadest (global).

The ordering for Razor Pages is somewhat simpler, given that you have only two scopes: global scope filters and Razor Page scope filters. For Razor Pages, global scope filters run the `*Executing` and `PageHandlerSelected` methods first, followed by the page scope filters. For the `*Executed` methods, the filters run in reverse order.

You'll sometimes find you need a bit more control over this order, especially if you have, for example, multiple action filters applied at the same scope. The filter pipeline caters to this requirement by way of the `IOrderedFilter` interface.

21.6.2 Overriding the default order of filter execution with IOrderedFilter

Filters are great for extracting cross-cutting concerns from your controller actions and Razor Page, but if you have multiple filters applied to an action, you'll sometimes need to control the precise order in which they execute.

Scope can get you some of the way, but for those other cases, you can implement `IOrderedFilter`. This interface consists of a single